

编程基础 III — Lambda演算

软件工程与计算 I · 2026 · 南京大学

| 刘钦

知识点

- **Lambda演算**的语法与归约规则
- **Church数**对自然数的编码
- **算术运算**：加、乘、幂、前驱、减法
- **逻辑运算**：TRUE / FALSE / AND / OR / NOT / IF
- **谓词**：ISZERO / LEQ / EQ / MAX / MIN
- **递归**问题及其解决思路
- **Y Combinator** 与不动点理论
- **有序对**：CONS / CAR / CDR / LENGTH

Outline

Lambda演算定义

- 语法三要素
- 自由变量与绑定变量
- β 归约 · 柯里化
- α 等价 · η 规约
- 求值策略（正规序 vs 应用序）

算术计算与逻辑谓词

- Church数 (ZERO / SUCC / ONE...)
- 算术: PLUS / MULT / POW / PRED / SUB
- 逻辑: TRUE / FALSE / AND / OR / NOT / IF
- 谓词: ISZERO / LEQ / EQ / MAX / MIN

递归与Y Combinator

- 无名函数的递归难题
- 多一个参数 / W组合子
- 不动点理论
- Y Combinator (Curry)
- 图灵不动点组合子

有序对

- CONS / CAR / CDR / NIL / NULL
- LENGTH 函数

【AI视角】

- Lambda演算 × Transformer 注意力
- 递归 × LLM 链式推理 (CoT)

Lambda演算定义

语法 · β 归约 · 求值策略

什么是 Lambda 演算？

Lambda 演算 (λ -calculus) 是 Alonzo Church 于 1930 年代提出的形式系统，用于研究函数定义、函数应用与递归。

- 图灵证明：Lambda 演算与图灵机在计算能力上**完全等价**
- 是**函数式编程** (Haskell、Lisp、ML) 的理论基础
- 与邱奇-图灵论题中的 μ -递归函数三者**等价**

Lambda 演算的语法

Lambda 表达式（项）由三种规则递归定义：

规则	语法	含义
变量	x	一个标识符
抽象	$\lambda x. M$	以 x 为参数、 M 为体的函数
应用	$M N$	将 M 应用于 N （即将 N 代入 M ）

约定

- 应用左结合： $M N P = (M N) P$
- 抽象右结合： $\lambda x. \lambda y. M = \lambda x. (\lambda y. M)$
- 缩写： $\lambda x. \lambda y. M$ 写作 $\lambda x y. M$

自由变量与绑定变量

在 $\lambda x. M$ 中, x 是**绑定变量** (bound variable); M 中不被任何 λ 捕获的变量是**自由变量** (free variable)。

示例

$\lambda x. x$	- x 是绑定变量
$\lambda x. y$	- y 是自由变量
$\lambda x. x y$	- x 绑定, y 自由
$\lambda x. \lambda y. x y$	- x, y 均绑定
$(\lambda x. x) y$	- 应用后 y 自由

为什么重要?

β 归约 $(\lambda x. M) N \rightarrow M[x:=N]$ 中, 需要将 N 代入 M 里所有 x 的**自由出现**。

若 N 含有自由变量与 M 中绑定变量**同名**, 需先 α 重命名, 避免**变量捕获**:

$(\lambda x. \lambda y. x) y$
→ 先重命名: $(\lambda x. \lambda z. x) y$
→ $\lambda z. y$ ✓ (而非 $\lambda y. y x$)

β 归约 (β-reduction)

将实参代入函数体，是 Lambda 演算中唯一的计算规则。

$$(\lambda x. M) N \longrightarrow M[x := N]$$

示例

$(\lambda x. x) y$	$\rightarrow y$	(恒等函数)
$(\lambda x. x x) y$	$\rightarrow y y$	
$(\lambda x. \lambda y. x) a b$	$\rightarrow a$	(返回第一个参数)
$(\lambda x. \lambda y. y) a b$	$\rightarrow b$	(返回第二个参数)
$(\lambda x. x + 1) 5$	$\rightarrow 5 + 1 = 6$	

多步归约

$(\lambda x. \lambda y. x y) f a$	
$= (\lambda y. f y) a$	(代入 $x = f$)
$= f a$	(代入 $y = a$)

柯里化 (Currying)

Lambda 演算中，所有函数都只接受一个参数。多参数函数通过返回函数来模拟。

多参数函数 → 嵌套单参数函数

$\text{add}(x, y) = x + y$

变为：

$\lambda x. \lambda y. x + y$

调用过程：

$(\lambda x. \lambda y. x + y) 3 5$
 $= (\lambda y. 3 + y) 5$
 $= 3 + 5 = 8$

柯里化的威力：偏应用

$\text{add3} \equiv (\lambda x. \lambda y. x + y) 3$
 $= \lambda y. 3 + y$

$\text{add3 } 5 = 8$
 $\text{add3 } 7 = 10$

只传入部分参数，得到一个新函数

在函数式编程中随处可见： `map (+1) [1,2,3]`

α 等价 与 η 规约

α 等价 (重命名)

绑定变量的名字不影响含义:

$$\lambda x. x \equiv_{\alpha} \lambda y. y \equiv_{\alpha} \lambda z. z$$

就像 $f(x) = x^2$ 和 $f(t) = t^2$ 是同一个函数

η 规约 (外延等价)

$$\lambda x. f x \rightarrow_{\eta} f$$

(当 x 在 f 中不自由出现时)

直觉: $\lambda x. f x$ 和 f 在行为上完全一致

正规形式: 无法再进行任何 β 归约的表达式, 即计算的**最终结果**。

η 规约：三层理解

第一层：去掉空壳

`\x. f x` 接受 `x` 后转交给 `f`——这正是 `f` 本身在做的事，多余的包装可以去掉。

```
# Python 中同样成立
g = lambda x: f(x)    # ← 等价于
g = f                 # ← η 规约后

map(lambda x: str(x), lst) # ← 等价于
map(str, lst)             # ← η 规约后 (point-free)
```

第二层：内涵 vs 外延

	相等标准
内涵	写法相同
外延	输入相同 → 输出相同

η 规约表达的是**外延等价**：行为完全一致的函数视为同一函数。

条件为何必要？

$$\lambda x. (x\ x) \neq (x\ x)$$

若 `x` 在 `f` 中自由出现，去掉 `\x.` 会改变语义——`x` 从绑定变量变成自由变量。

第三层：η 展开是 Z Combinator 的关键

`x x` → `\v. x x v` (η 展开)：把"立即求值"变成"等待调用"，让递归在惰性求值语言中也能终止。

求值策略

Church–Rosser 定理（合流性）：若 M 可归约到 N_1 和 N_2 ，则存在 P 使得 N_1 、 N_2 都能归约到 P 。

推论：正规形式（最终结果）若存在，则唯一。

不同归约顺序可能到达相同结果——但**效率和终止性**仍可能不同：

正规序（Normal Order）

最外层、最左边的 redex 优先

```
(\x. x x) ((\y. y) z)
→ ((\y.y) z) ((\y.y) z) ← 先展开外层
→ z ((\y.y) z)
→ z z
```

- 若有正规形，**必能到达**
- 可能重复计算参数

应用序（Applicative Order）

最内层、最左边的 redex 优先（先求值参数）

```
(\x. x x) ((\y. y) z)
→ (\x. x x) z ← 先化简参数
→ z z
```

- Haskell：惰性（正规序变体）
- Python / Java：应用序（call-by-value）
- Y Combinator 在应用序下**会无限循环**

应用序 = Call-by-Value: 为什么 Y 在 Python 中死循环

应用序的规则：先把参数算完，再代入函数体

应用序下展开 Y F

$Y F = F(Y F)$ ← 参数位置出现了 Y F 本身

应用序要求先把参数 Y F 求值完，但展开后又是 F(Y F)，陷入死循环：

```
Y F
→ F (Y F)
→ F (F (Y F))
→ F (F (F (Y F)))
→ ... ∞
```

每次使用 Y F 之前都得先展开它，永无止境。

应用序与 Ω ：为什么惰性求值安全

与 Ω 的关系

Y 内部含有 `(\x. g(x x))(\x. g(x x))`，结构上等同于：

```

$$\begin{aligned}\Omega_g &\equiv (\lambda x. g(x x))(\lambda x. g(x x)) \\ &\rightarrow g ((\lambda x. g(x x))(\lambda x. g(x x))) \\ &\rightarrow g (g (\dots))\end{aligned}$$

```

应用序先对 `x x` 求值 $\rightarrow$ 触发 Ω 发散。

惰性求值为何安全

正规序先代入 `g`，`x x` 还没展开就进入 `g` 的函数体，`g` 可以选择是否真正需要递归——由基准情形控制终止。

补充：Python 实际是 call-by-object-reference（传对象引用），但对求值时机而言是急性的——参数在进入函数体前一定已经求值完毕，这是与应用序对应的关键性质。

算术计算与逻辑谓词

Church数 · PLUS · MULT · POW · TRUE/FALSE · ISZERO

"模拟"自然数 — Church 数

Church 数将自然数 n 编码为**「将函数 f 作用于 x 共 n 次」**的高阶函数

```
ZERO  ≡ \f.\x. x
SUCC  ≡ \n.\f.\x. f (n f x)
ONE   ≡ \f.\x. f x
TWO   ≡ \f.\x. f (f x)
THREE ≡ \f.\x. f (f (f x))
FOUR  ≡ \f.\x. f (f (f (f x)))
```

直觉: $n \ f \ x = f$ 应用 n 次于 x

```
TWO succ 0 → succ (succ 0) = 2
THREE succ 0 → succ(succ(succ 0)) = 3
```

验证 **SUCC ONE = TWO**

```
SUCC ONE
= (\n.\f.\x. f (n f x)) (\f.\x. f x)
= \f.\x. f ((\f.\x. f x) f x)
= \f.\x. f ((\x. f x) x)
```

加法：PLUS

$$\text{PLUS} \equiv \lambda m.\lambda n.\lambda f.\lambda x. m \ f \ (n \ f \ x)$$

直觉：先将 f 作用 n 次，再作用 m 次，共 $m+n$ 次。

先看简单情形： PLUS ONE TWO

$$\begin{aligned} \text{PLUS ONE TWO} &= \lambda f.\lambda x. \text{ONE } f \ (\text{TWO } f \ x) \\ &= \lambda f.\lambda x. \text{ONE } f \ (f \ (f \ x)) \\ &= \lambda f.\lambda x. f \ (f \ (f \ x)) \\ &= \text{THREE} \quad \checkmark \end{aligned}$$

推导 PLUS TWO THREE

$$\begin{aligned} \text{PLUS TWO THREE} &= (\lambda m.\lambda n.\lambda f.\lambda x. m \ f \ (n \ f \ x)) \ \text{TWO THREE} \\ &= \lambda f.\lambda x. \text{TWO } f \ (\text{THREE } f \ x) \\ &= \lambda f.\lambda x. \text{TWO } f \ (f \ (f \ (f \ x))) \\ &= \lambda f.\lambda x. f \ (f \ (f \ (f \ (f \ x)))) \\ &= \text{FIVE} \quad \checkmark \end{aligned}$$

乘法：MULT

$MULT \equiv \lambda m. \lambda n. \lambda f. m (n f)$

直觉：将「f 应用 n 次」这件事本身应用 m 次，共 $m \times n$ 次。

先看简单情形： **MULT TWO TWO**

```
MULT TWO TWO
= \f. TWO (TWO f)
= \f.\x. TWO (TWO f) x
= \f.\x. (TWO f) ((TWO f) x)
= \f.\x. (TWO f) (f (f x))
= \f.\x. f (f (f (f x)))
= FOUR    ✓
```

推导 **MULT THREE TWO**

```
MULT THREE TWO
= \f. THREE (TWO f)
= \f.\x. (TWO f) ((TWO f) ((TWO f) x))
= \f.\x. f(f( f(f( f(f(x))))))
= SIX    ✓
```

幂：POW

$POW \equiv \lambda b. \lambda e. e \ b$

直觉：e 次方 = 将「乘以 b」这件事应用 e 次。

推导 **POW TWO TWO** (第一步： β 归约)

$POW \ TWO \ TWO$	
$= TWO \ TWO$	(β 代入 $b=TWO, e=TWO$)
$= (\lambda f. \lambda x. f(f \ x)) \ TWO$	(展开 $e = TWO$)
$= \lambda x. TWO \ (TWO \ x)$	($f \leftarrow TWO$, 还差 f 和 x 才能看出等于 $FOUR$)

Church 数含义： $\lambda x. TWO(TWO \ x)$ 是一个等待 f 和 x 的函数。下页验证它等于 $FOUR$ 。

幂：POW — 验证 TWO TWO = FOUR

对任意 f 、 x ，验证 $(\text{TWO TWO}) f x = f(f(f(f x)))$ ：

```
(TWO TWO) f x
= TWO (TWO f) x      (代入 \x. TWO(TWO x), x 位置填入 f, 再作用于 x)
= (TWO f) ((TWO f) x) (TWO g z = g(g z), 令 g = TWO f, z = x)
= (TWO f) (f (f x))   (TWO f x = f(f x))
= f (f (f (f x)))     (TWO f (f(fx)) = f(f(f(fx))))
= FOUR f x    ✓    (22 = 4)
```

推导 POW TWO THREE

```
POW TWO THREE
= THREE TWO
= TWO TWO TWO
= EIGHT    ✓    (23 = 8)
```

前驱：PRED

Church 数的减一是最难的操作——0 无前驱，且 SUCC 的逆不直接存在。

$$\text{PRED} \equiv \lambda n. \lambda f. \lambda x. n (\lambda g. \lambda h. h (g f)) (\lambda u. x) (\lambda u. u)$$

构造直觉：「滑动窗口」

维护一个对 (prev, curr)，每步向前滑动：从 (·, 0) 出发，经过 n 步变为 (n-1, n)，取第一个元素即 n-1。

初始对：	($\lambda u. x$)	– 相当于 ($_, 0$)
滑动步：	$\lambda g. \lambda h. h (g f)$	– (prev, curr) $\rightarrow$ (curr, f curr)
取结果：	$\lambda u. u$	– 取对中的"当前值"函数

PRED ONE $\leftarrow$ 滑动 1 次：($_, 0$) $\rightarrow$ (0, 1) $\rightarrow$ 取前 = 0 = ZERO ✓

PRED TWO $\leftarrow$ 滑动 2 次：($_, 0$) $\rightarrow$ (0, 1) $\rightarrow$ (1, 2) $\rightarrow$ 取前 = 1 = ONE ✓

减法：SUB

$\text{SUB} \equiv \backslash m. \backslash n. n \text{ PRED } m$

直觉：将 PRED 作用 n 次于 m ，即 $m - n$ （若 $n > m$ 则截止到 ZERO）。

SUB FIVE TWO
= TWO PRED FIVE
= PRED (PRED FIVE)
= PRED FOUR
= THREE ✓

SUB TWO FIVE
= FIVE PRED TWO
= PRED(PRED(PRED(PRED(PRED TWO))))
= ZERO ✓ （截断减法，不为负）

"模拟"逻辑

TRUE 和 FALSE 被编码为**选择函数**：从两个选项中取一个

TRUE $\equiv \backslash x.\backslash y. x$ (返回第一个参数)

FALSE $\equiv \backslash x.\backslash y. y$ (返回第二个参数)

AND $\equiv \backslash p.\backslash q. p q p$

OR $\equiv \backslash p.\backslash q. p p q$

NOT $\equiv \backslash p. p \text{ FALSE TRUE}$

IF $\equiv \backslash p.\backslash a.\backslash b. p a b$

直觉： $\text{IF } p a b = p a b$ ——TRUE 返回第一个选项，
FALSE 返回第二个。

验证 AND TRUE FALSE

AND TRUE FALSE
= TRUE FALSE TRUE
= $(\backslash x.\backslash y. x) \text{ FALSE TRUE}$
= FALSE ✓

验证 OR FALSE TRUE

OR FALSE TRUE
= FALSE FALSE TRUE
= $(\backslash x.\backslash y. y) \text{ FALSE TRUE}$
= TRUE ✓

"模拟"逻辑 — NOT 与 IF

验证 NOT FALSE

```
NOT FALSE
= FALSE FALSE TRUE
= (\x.\y. y) FALSE TRUE
= TRUE ✓
```

验证 IF TRUE a b

```
IF TRUE a b
= TRUE a b
= (\x.\y. x) a b
= a ✓
```

验证 NOT TRUE

```
NOT TRUE
= TRUE FALSE TRUE
= (\x.\y. x) FALSE TRUE
= FALSE ✓
```

验证 IF FALSE a b

```
IF FALSE a b
= FALSE a b
= (\x.\y. y) a b
= b ✓
```

"模拟"谓词

$ISZERO \equiv \lambda n. n (\lambda x. FALSE) TRUE$

直觉：若 $n=0$ ，不应用任何函数，返回 TRUE；否则反复替换为 FALSE。

$ISZERO\ ZERO = ZERO (\lambda x. FALSE) TRUE = TRUE \quad \checkmark$

$ISZERO\ ONE = ONE (\lambda x. FALSE) TRUE$
 $= (\lambda x. FALSE) TRUE = FALSE \quad \checkmark$

$ISZERO\ TWO = TWO (\lambda x. FALSE) TRUE$
 $= (\lambda x. FALSE)((\lambda x. FALSE) TRUE)$
 $= (\lambda x. FALSE) FALSE = FALSE \quad \checkmark$

$LEQ \equiv \lambda m. \lambda n. ISZERO (SUB\ m\ n)$

$EQ \equiv \lambda m. \lambda n. AND (LEQ\ m\ n) (LEQ\ n\ m)$

$MAX \equiv \lambda m. \lambda n. IF (LEQ\ m\ n) n\ m$

$MIN \equiv \lambda m. \lambda n. IF (LEQ\ m\ n) m\ n$

验证 **LEQ TWO THREE**

$LEQ\ TWO\ THREE$
 $= ISZERO (SUB\ TWO\ THREE)$
 $= ISZERO (THREE\ PRED\ TWO)$
 $\quad (PRED(PRED(PRED\ TWO)))$ 截止到 ZERO)
 $= ISZERO\ ZERO$
 $= TRUE \quad \checkmark$

递归与Y Combinator

无名函数的递归 · 不动点 · Y Combinator · 图灵组合子

正规形式与发散

并非所有 Lambda 项都能到达正规形式——有些项会永远归约下去。

Ω 组合子（最简发散例）

$$\Omega \equiv (\lambda x. x x) (\lambda x. x x)$$

$$\Omega \rightarrow (\lambda x. x x) (\lambda x. x x) \rightarrow (\lambda x. x x) (\lambda x. x x) \rightarrow \dots$$

无论归约多少步，永远得到自身——**无正规形式，无限循环**。

为什么重要？

Y Combinator 的核心结构 $(\lambda x. g(x x))(\lambda x. g(x x))$ 与 Ω 类似：

- 在**惰性求值**（正规序）下：外层的 **g** 先展开，递归有机会终止
- 在**急性求值**（应用序）下：先对参数求值，遇到 **x x** 就无限展开，**死循环**

递归的难题

在 Lambda 演算中，函数**没有名字**。如何表达「函数调用自身」？

我们希望定义阶乘：

```
FACT n = if n == 0 then 1 else n * FACT(n-1)
```

但在 Lambda 演算中，`FACT` 在定义时**尚未存在**，不能直接引用。

方法一：多一个参数

将函数本身作为参数传入——

```
FACT2 ≡ \f.\n. IF (ISZERO n) ONE (MULT n (f f (PRED n)))  
FACT  ≡ FACT2 FACT2
```

验证：

```
FACT THREE  
= FACT2 FACT2 THREE  
= IF (ISZERO THREE) ONE (MULT THREE (FACT2 FACT2 TWO))  
= MULT THREE (MULT TWO (MULT ONE ONE))  
= SIX    ✓
```

W 组合子 (Mockingbird)

$$W \equiv \lambda x. x x$$

将「把自身传给自己」这个动作抽象出来：

$$W \text{ FACT2} = \text{FACT2} \quad \text{FACT2} = \text{FACT}$$

W 组合子揭示了递归的本质：自我应用 (self-application)

这还不够优雅——能否从 F 直接得到它的不动点？

不动点 (Fixed Point)

数学中的不动点

若 $F(x) = x$ ，则 x 是 F 的不动点。

- `cos` 函数：不动点 $\approx 0.739085\dots$
- 恒等函数 $f(x)=x$ ：每个 x 都是不动点

递归函数的不动点

令

```
F ≡ \f.\n. IF (ISZERO n) ONE  
          (MULT n (f (PRED n)))
```

则 $FACT = F FACT$

即 $FACT$ 是 F 的不动点！

关键洞见：如果能找到一个算子 Y ，使得对任意 F 都有 $Y F = F (Y F)$ ，就能自动实现递归。

Y Combinator (Curry 不动点组合子)

$$Y \equiv \lambda g. (\lambda x. g (x x)) \lambda x. g (x x)$$

核心性质: $Y F = F (Y F)$

证明

$$\begin{aligned} Y F &= (\lambda g. (\lambda x. g (x x)) \lambda x. g (x x)) F \\ &= (\lambda x. F (x x)) \lambda x. F (x x) \\ &= F ((\lambda x. F (x x)) \lambda x. F (x x)) \\ &= F (Y F) \end{aligned}$$

$Y F$ 是 F 的不动点。

用 Y Combinator 实现阶乘

定义递归体（不含自引用）：

```
R ≡ \f.\n. IF (ISZERO n) ONE (MULT n (f (PRED n)))
```

则 `FACT = Y R`，因为 `Y R = R (Y R)` 正好满足阶乘的递归结构。

`Y R THREE` 的归约过程 ($3! = 6$)

```
Y R THREE
= R (Y R) THREE
= IF (ISZERO THREE) ONE (MULT THREE ((Y R) TWO))
= MULT THREE (Y R TWO)           (ISZERO THREE = FALSE)
= MULT THREE (R (Y R) TWO)
= MULT THREE (MULT TWO (Y R ONE))
= MULT THREE (MULT TWO (MULT ONE (Y R ZERO)))
= MULT THREE (MULT TWO (MULT ONE ONE)) (ISZERO ZERO = TRUE → ONE)
= MULT THREE (MULT TWO ONE)
= MULT THREE TWO
= SIX    ✓    (3! = 6)
```

图灵不动点组合子

除了 Y Combinator, Alan Turing 也发现了另一种不动点组合子:

$$\top \equiv (\lambda x. \lambda y. y (x \times y)) \ \lambda x. \lambda y. y (x \times y)$$

```
Lambda> T = (\x.\y.y (x x y)) \x.\y.y (x x y)
```

```
Lambda> FACTT = T FACT2
```

```
Lambda> FACTT FOUR
```

```
\f.\x.f (f (f (f (f (f (f (f (f (f (f (f (f (f (f (f (f (f
```

FOUR! = 24, Church 数表示为 24 次函数应用 ✓

Y Combinator 对比图灵组合子

Y Combinator (Curry)

$$Y \equiv \lambda g. (\lambda x. g(x \ x)) \lambda x. g(x \ x)$$

- 惰性求值语言中**直接可用**
- 急性求值语言中会**无限展开**（遇到 `x x` 立即求值 $\rightarrow \Omega$ 发散）
- 需要引入 η 展开：Z Combinator

图灵不动点组合子

$$T \equiv (\lambda x. \lambda y. y(x \ x \ y)) \lambda x. \lambda y. y(x \ x \ y)$$

- 在急性求值下**也能工作**
- 对应图灵的原始构造

两者都满足不动点性质，实现了 Lambda 演算中的**递归**。

Z Combinator (急性求值下的 Y)

在 Python、Java 等急性求值语言中，Y Combinator 的 `x x` 会立即展开并死循环。解决方案：用 η 展开将 `x x` 包裹在一个 λ 下，延迟求值。

Y Combinator (惰性)

$$Y \equiv \lambda g. (\lambda x. g (x x)) \\ \lambda x. g (x x)$$

- `x x` 直接出现
- 急性求值下立即展开 $\rightarrow \infty$

Z Combinator (急性)

$$Z \equiv \lambda g. (\lambda x. g (\lambda v. x x v)) \\ \lambda x. g (\lambda v. x x v)$$

- `x x` 被 `\v.` 包住，变成值
- 只有真正调用时才展开
- Python / Scheme (非惰性) 可用

Python 中用 Z Combinator 实现阶乘 (无名递归)

```
Z = lambda g: (lambda x: g(lambda v: x(x)(v)))(lambda x: g(lambda v: x(x)(v)))
fact = Z(lambda f: lambda n: 1 if n == 0 else n * f(n - 1))
fact(5) # -> 120
```

有序对

CONS · CAR · CDR · NIL · NULL · LENGTH

还记得有序对吗？

在数据抽象中，我们用 `make-rat / numer / denom` 表示有理数；用 `cons / car / cdr` 表示链表.....

这些操作能否完全用 Lambda 演算来定义？

构建有序对

$\text{CONS} \equiv \lambda x.\lambda y.\lambda f. f\ x\ y$ (构造: 将 x, y 存入)

$\text{CAR} \equiv \lambda p. p\ \text{TRUE}$ (取第一个元素)

$\text{CDR} \equiv \lambda p. p\ \text{FALSE}$ (取第二个元素)

$\text{NIL} \equiv \lambda x. \text{TRUE}$ (空表)

$\text{NULL} \equiv \lambda p. p\ (\lambda x.\lambda y. \text{FALSE})$ (判断是否为空)

验证 $\text{CAR} (\text{CONS}\ a\ b)$

$\text{CAR} (\text{CONS}\ a\ b)$

$= (\lambda p. p\ \text{TRUE}) ((\lambda x.\lambda y.\lambda f. f\ x\ y)\ a\ b)$

$= (\lambda p. p\ \text{TRUE}) (\lambda f. f\ a\ b)$

$= (\lambda f. f\ a\ b)\ \text{TRUE}$

$= \text{TRUE}\ a\ b = (\lambda x.\lambda y. x)\ a\ b = a \quad \checkmark$

验证 $\text{CDR} (\text{CONS}\ a\ b)$

$\text{CDR} (\text{CONS}\ a\ b)$

$= (\lambda p. p\ \text{FALSE}) (\lambda f. f\ a\ b)$

$= (\lambda f. f\ a\ b)\ \text{FALSE}$

$= \text{FALSE}\ a\ b = (\lambda x.\lambda y. y)\ a\ b = b \quad \checkmark$

验证基本操作

```
Lambda> CONS a (CONS b (CONS c NIL))
```

```
\f.f a \f.f b \f.f c \x.\y.x
```

```
Lambda> CAR (CONS a (CONS b (CONS c NIL)))
```

```
a
```

```
Lambda> CDR (CONS a (CONS b (CONS c NIL)))
```

```
\f.f b \f.f c \x.\y.x
```

```
Lambda> CAR (CDR (CONS a (CONS b (CONS c NIL))))
```

```
b
```

```
Lambda> NULL (CDR (CONS a (CONS b (CONS c NIL))))
```

```
\x.\y.y      (= FALSE, 非空)
```

```
Lambda> NULL NIL
```

```
\x.\y.x      (= TRUE, 为空)
```

定义 LENGTH 函数

结合 Y Combinator 与有序对操作：

```
LENGTH ≡ Y (\g.\c.\x. NULL x c (g (SUCC c) (CDR x))) ZERO
```

直觉：从计数器 0 开始，若表非空则对尾递归并加一，直到空表返回计数。

测试

```
Lambda> LENGTH NIL  
\f.\x.x          (= ZERO)
```

```
Lambda> LENGTH (CONS a (CONS b (CONS c NIL)))  
\f.\x.f (f (f x))    (= THREE)
```

```
Lambda> LENGTH (CONS a (CONS b (CONS c (CONS d NIL))))  
\f.\x.f (f (f (f x))) (= FOUR)
```

总结

本讲总结

Lambda 演算

概念	定义
抽象	<code>\x.M</code> — 函数定义
应用	<code>M N</code> — 函数调用
β 归约	<code>(\x.M) N $\rightarrow$ M[x:=N]</code>

Church 数

数	编码
ZERO	<code>\f.\x. x</code>
SUCC	<code>\n.\f.\x. f(n f x)</code>
n	f applied n times to x

运算一览

运算	关键思路
PLUS m n	f 先作用 n 次再 m 次
MULT m n	将「n 次」重复 m 遍
POW b e	将「xb」重复 e 遍
TRUE/FALSE	选择函数
ISZERO	零次替换保留 TRUE
Y Combinator	<code>Y F = F(Y F)</code>
CONS/CAR/CDR	函数作为数据容器

核心洞见回顾

Church 数：自然数 = 「函数应用次数」。纯函数即可表达所有算术。

逻辑：TRUE / FALSE = 选择函数。 `IF p a b` 就是 `p a b`。

Y Combinator： `Y F = F(Y F)` ——无需名字即可递归。任何递归函数都是某个"递归体"的不动点。

有序对： `CONS x y = \f. f x y` ——函数可以存储数据，数据结构不过是「等待被选择的函数」。

【AI视角】Lambda演算 ↔ Attention：抽象/应用/ β -归约对应 Q/K/V 加权组合；两者都是在上下文中查找并组合信息。

【AI视角】递归 ↔ CoT：Chain-of-Thought 让 LLM 模拟递归的"分解→子问题→合并"思维模式。

课后练习

推导 · 验证 · 深入思考

练习 1 — 归约练习

手动完成以下 β 归约，写出每一步：

(1) $(\lambda x. \lambda y. y\ x)\ a\ b$

(2) $(\lambda f. f\ f)\ (\lambda x. x)$

(3) $(\lambda x. \lambda y. \lambda z. x\ z\ (y\ z))\ (\lambda a. \lambda b. a)\ (\lambda a. \lambda b. b)\ c$

提示：(3) 是 SKI 组合子演算中的 S 组合子应用——尝试推导出结果。

练习 2 — Church 数验证

验证以下等式（通过 β 归约推导）：

乘法

MULT TWO THREE = ?

(应等于 SIX = `\f.\x.f(f(f(f(f x))))`)

幂

POW THREE TWO = ?

(应等于 NINE)

前驱

PRED ONE = ?

PRED TWO = ?

(应分别等于 ZERO 和 ONE)

减法

SUB FOUR TWO = ?

SUB TWO FOUR = ?

练习 3 — 逻辑运算推导

验证以下逻辑等式（通过 β 归约）：

表达式	期望结果
NOT TRUE	FALSE
OR FALSE FALSE	FALSE
OR TRUE FALSE	TRUE
AND TRUE TRUE	TRUE
IF FALSE a b	b

回忆： $\text{TRUE} \equiv \lambda x.\lambda y. x$, $\text{FALSE} \equiv \lambda x.\lambda y. y$

练习 4 — Y Combinator 的另一面

考察以下函数（Fibonacci 数的递归体）：

```
FIB_BODY ≡ \f.\n. IF (LEQ n ONE) n  
                (PLUS (f (PRED n)) (f (SUB n TWO)))
```

1. 写出 `Y FIB_BODY` 对应的函数，它计算什么？
2. 追踪 `(Y FIB_BODY) THREE` 的归约过程（可以简化步骤）
3. 为什么在**急性求值**（applicative-order）语言中直接使用 Y Combinator 会出现问题？该如何修改？

练习 5 — 从有序对到链表

已有 `CONS / CAR / CDR / NIL / NULL / LENGTH`，尝试用 Lambda 演算定义：

1. APPEND

将两个列表拼接：

```
APPEND (CONS a NIL) (CONS b NIL)
= CONS a (CONS b NIL)
```

提示：用 Y Combinator 递归遍历第一个列表。

2. MAP

将函数应用到列表每个元素：

```
MAP SUCC (CONS ONE (CONS TWO NIL))
= CONS TWO (CONS THREE NIL)
```

提示：结构与 APPEND 类似，区别在于对每个元素调用 f。

【AI视角】Lambda演算 × Transformer 注意力

Lambda 演算的核心

- **抽象** ($\lambda x.M$): 绑定变量, 推迟求值
- **应用** ($M\ N$): 将函数作用于参数
- **β -归约**: 逐步展开计算

```
(λf. λx. f (f x)) succ zero  
→ (λx. succ (succ x)) zero  
→ succ (succ zero)  
→ TWO
```

Transformer Attention 的对应

- **Query** (Q) ↔ 抽象的"函数位"
- **Key-Value** (KV) ↔ 参数和绑定环境
- **Softmax(QK^T)V** ↔ 加权"应用"

```
Attention(Q,K,V)  
= softmax(QKT/√d) · V
```

两者本质都是: 在上下文中查找并组合信息。

Lambda 演算证明了任何计算都可用函数和应用表达; Transformer 则学会了在高维空间中执行类似的"查询-组合"操作。

【AI视角】递归 × LLM 的链式推理

递归的本质

```
(define (fact n)
  (if (= n 0) 1
      (* n (fact (- n 1)))))
```

- **基础情况**：直接给出答案
- **递归步骤**：将问题规模减小
- **信任子问题**：不必关心子调用如何实现

Chain-of-Thought Prompting

Q: 计算 5!

→ $5! = 5 \times 4!$

→ $4! = 4 \times 3!$

→ $3! = 3 \times 2!$

→ $2! = 2 \times 1! = 2$

→ 合并: $5 \times 4 \times 3 \times 2 \times 1 = 120$

- **分解子问题** ↔ 递归调用
- **逐步推导** ↔ 归约序列
- **合并结果** ↔ 返回栈展开

Chain-of-Thought 让 LLM 模拟递归思维；理解递归，就是理解 LLM 最有效的推理模式。

编程基础 III — Lambda演算

To understand recursion, you must first understand recursion.

— 程序员谚语